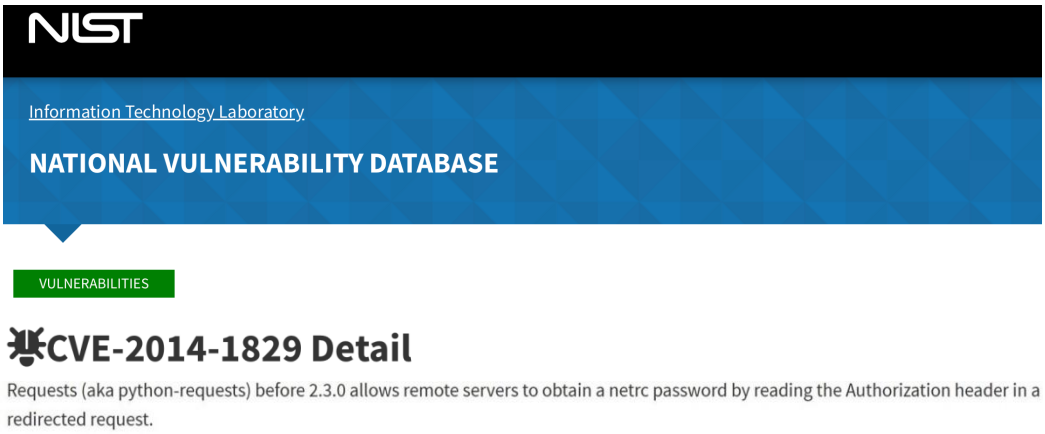
========= *Merge* Relationship =========

1 The example vulnerability and its patches

1.1 The description for the vulnerability

This example is a vulnerability item (CVE-2014-1829), which reports a vulnerability in the *python-requests* HTTP library before version 2.3.0. The flaw allows remote servers to obtain sensitive *netrc* passwords by exploiting improper handling of the *Authorization* header during HTTP redirects. When a request using credentials from the *netrc* file is redirected by a malicious or compromised server, the affected versions of the library incorrectly forward the *Authorization* header containing the plaintext password to the redirected target. Its CVE description is as follows:



1.2 The patches of this vulnerability

As indicated by the dataset of a top-tier paper[1], Commit *7ba5a53*, Commit *326a22e*, Commit *d9f34c6*, and Commit *f1893c8* are four patches of this vulnerability:

- ***7ba5a53****4ae9fc24e40b3ae6c480c9075d684727e*
- ***326a22e****8880c1dba52698a479eb7b6038d5b2e87*
- ***d9f34c6****848b9b313beefa7d3ce05f52fdea28c27*
- ***f1893c8****35570d72823c970fbd6e0e42c13b1f0f2*

2 The Relationship among Patches

The first three Patches (***7ba5a53, 326a22e, d9f34c6***) represent intermediate commits that were developed along the same feature branch during the vulnerability fixing process, as shown in Figure 1 ~ Figure 3.

```
Repopulate ~/.netrc auth.
master · v3.0.0a1

Filter files...
requests
  sessions.py

1 file changed +17 -0 lines changed
requests/sessions.py
@@ -158,6 +158,23 @@ def resolve_redirects(self, resp, req, stream=False, timeout=None,
158 158             prepared_request._cookies.update(self.cookies)
159 159             prepared_request.prepare_cookies(prepared_request._cookies)
160 160
    161 +           # If we get redirected to a new host, we should strip out any
    162 +           # authentication headers.
    163 +           original_parsed = urlparse(resp.request.url)
    164 +           redirect_parsed = urlparse(url)
    165 +
    166 +           if original_parsed.hostname != redirect_parsed.hostname:
    167 +               try:
    168 +                   del headers['Authorization']
    169 +               except KeyError:
    170 +                   pass
    171 +
    172 +           # However, .netrc might have more auth for us. Let's get it if it
    173 +           # does.
    174 +           new_auth = get_netrc_auth(url)
    175 +           if new_auth is not None:
    176 +               prepared_request.prepare_auth(new_auth)
    177 +
```

Figure 1. the Contents of Patch ***7ba5a53***

```
Better layout for checking.
master · v3.0.0a1

Filter files...
requests
  sessions.py

1 file changed +3 -5 lines changed
requests/sessions.py
@@ -163,11 +163,9 @@ def resolve_redirects(self, resp, req, stream=False, timeout=None,
163 163             original_parsed = urlparse(resp.request.url)
164 164             redirect_parsed = urlparse(url)
165 165
166     -           if original_parsed.hostname != redirect_parsed.hostname:
167     -               try:
168     -                   del headers['Authorization']
169     -               except KeyError:
170     -                   pass
    166 +           if (original_parsed.hostname != redirect_parsed.hostname and
    167 +                   'Authorization' in headers):
    168 +               del headers['Authorization']
```

Figure 2. the Contents of Patch ***326a22e***

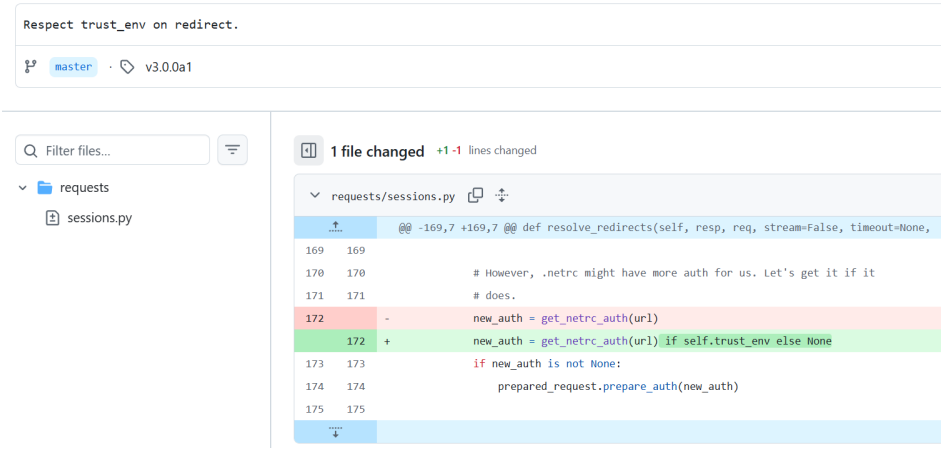


Figure 3. the Contents of Patch ***d9f34c6***

The changes of the above three patches are later consolidated by the last Patch f1893c8 through a branch merge, as shown in Figure 4. The presence of the term "Merge" in its description also clearly indicates that it is a merge commit. Structurally, it has two parent commits, which is a typical characteristic of Git merge commits and indicates that the histories of two development branches are combined in this commit.

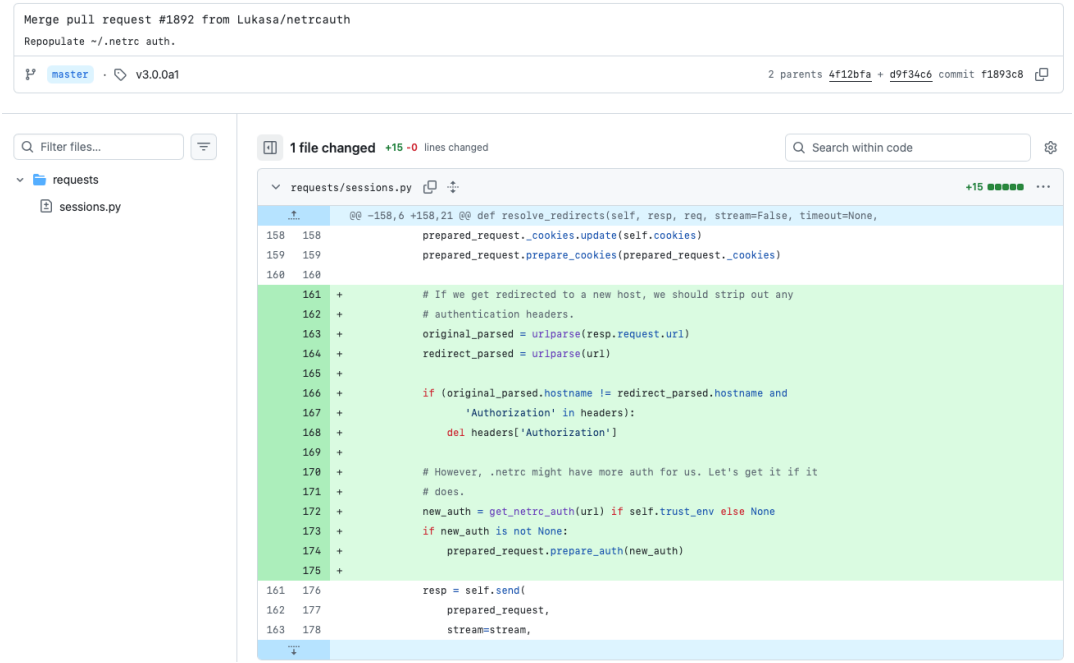


Figure 4. the Contents of Patch ***f1893c8***

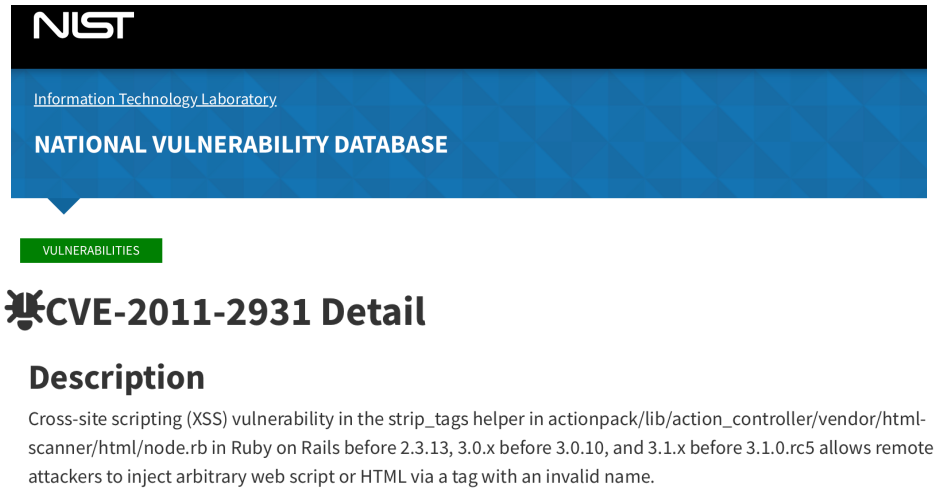
Thus, we label the relationship between the first three patches and the last patch of this CVE as *Merge*.

========= *Mirror* Relationship =========

1 The example vulnerability and its patches

1.1 The description for the vulnerability

This example is a vulnerability item (CVE-2011-2931), which reports a cross-site scripting (XSS) vulnerability discovered in certain versions of *Ruby* on *Rails*. This flaw allows remote unauthenticated attackers to inject arbitrary web scripts or HTML content into vulnerable web applications built with the framework. The description is as follows:



1.2 The patches of this vulnerability

As indicated by the dataset of a top-tier paper[1], Commit *3480d97* and Commit *586a944* are two patches of this vulnerability:

- ***3480d97****b6c9f657ca1d0f11ac1e3e17baf84cdb2*
- ***586a944****ddd4d03e66dea1093306147594748037a*

2 The Relationship between Patches

The former Patch ***3480d97*** is first applied to the *3-0-stable* branch to strip tags with invalid names as shown in Figure 5, and then the latter Patch ***586a944*** with the same code change is submitted to the *master* branch as illustrated in Figure 6.

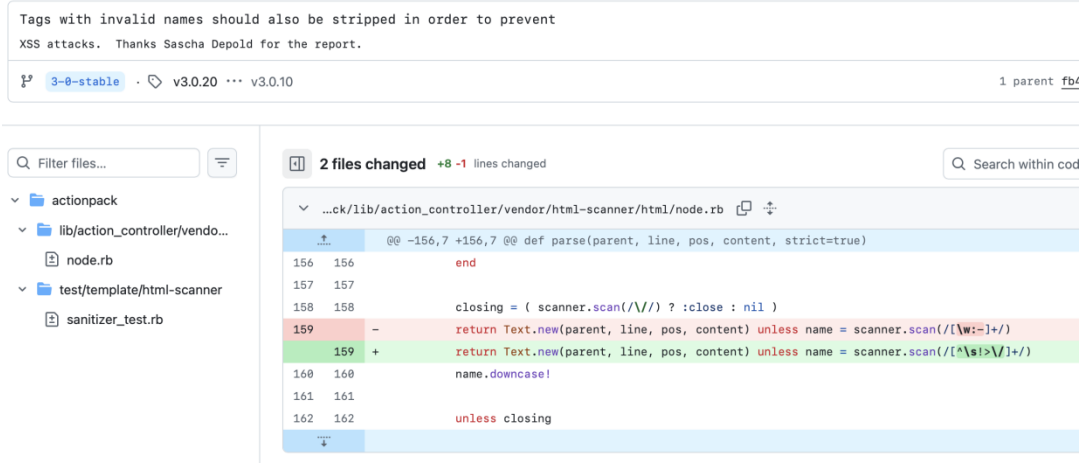


Figure 5. the Contents of Patch ***3480d97***

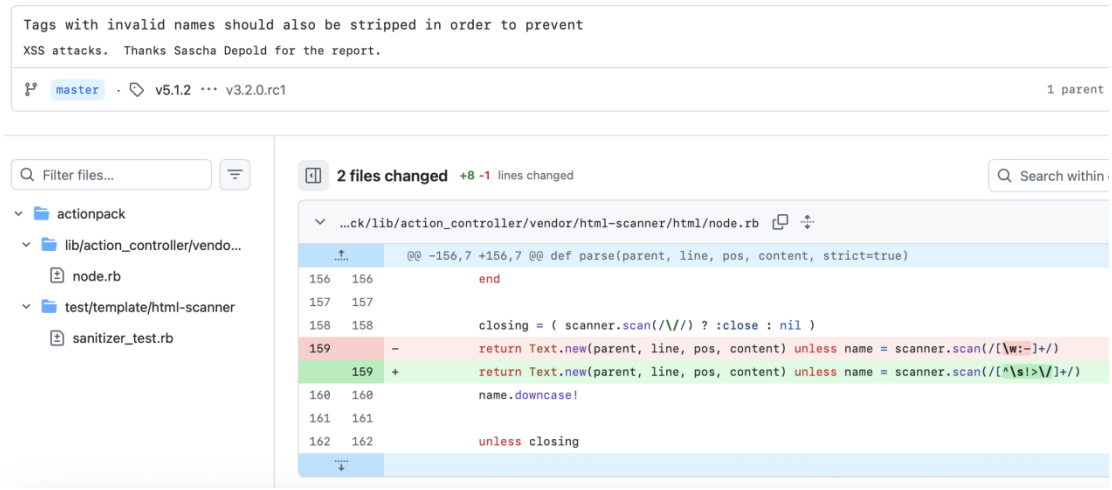


Figure 6. the Contents of Patch ***586a944***

Although the two patches contain identical changes and address the same vulnerability, they are committed independently to different branches, therefore belong to independent code evolution paths with distinct commit histories and parent pointers.

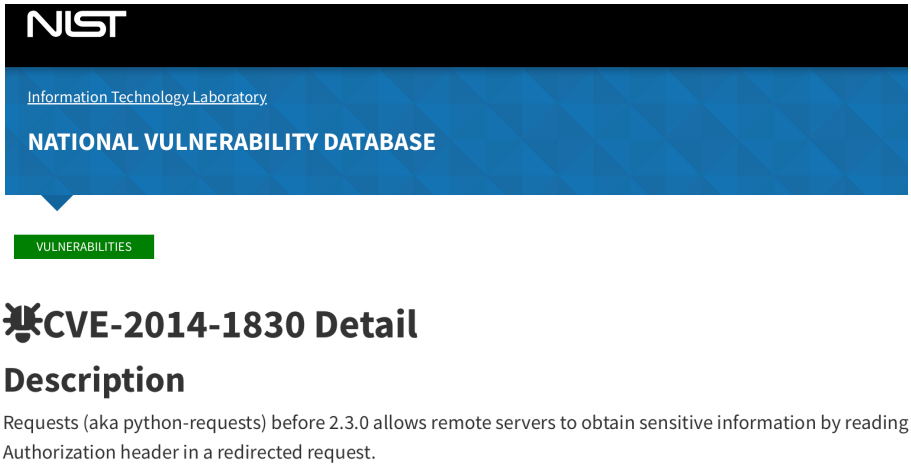
Thus, we label the relationship between the two patches of this CVE as *Mirror*.

========= *Better Solution* Relationship =========

1 The example vulnerability and its patches

1.1 The description for the vulnerability

This example is a vulnerability item (CVE-2014-1830), which reports a security flaw that allows malicious remote servers to obtain sensitive user credentials by reading the *Proxy-Authorization* header during HTTP request redirection. When a vulnerable client follows a server-initiated redirect, it incorrectly forwards the *Proxy-Authorization* header containing authentication information to the target host, thereby exposing sensitive data to unauthorized parties. The description is as follows:



1.2 The patches of this vulnerability

As indicated by the dataset of a top-tier paper[1], Commit *7ba5a53* and Commit *326a22e* are two patches of this vulnerability:

- ***7ba5a53***4ae9fc24e40b3ae6c480c9075d684727e
- ***326a22e***8880c1dba52698a479eb7b6038d5b2e87

2 The Relationship between Patches

Patch ***7ba5a53*** fixes the vulnerability by removing authentication headers when redirects cross different hosts, thereby preventing sensitive authentication information from being exposed during redirection, as shown in Figure 7. A subsequent Patch ***326a22e*** builds directly upon the logic

introduced in Patch ***7ba5a53*** and does not modify the underlying security mechanism that prevents credential leakage during redirects, but improves the implementation. Instead of using a try-except block to remove the *Authorization* header, it introduces a clearer conditional check ("*Authorization*" *in headers*) before deleting the header, as shown in Figure 8.

```
Repopulate ~/.netrc auth.
master · v3.0.0a1

Filter files...
requests
  sessions.py

1 file changed +17 -0 lines changed
requests/sessions.py
@@ -158,6 +158,23 @@ def resolve_redirects(self, resp, req, stream=False, timeout=None,
158 158             prepared_request._cookies.update(self.cookies)
159 159             prepared_request.prepare_cookies(prepared_request._cookies)
160 160
    161 +           # If we get redirected to a new host, we should strip out any
    162 +           # authentication headers.
    163 +           original_parsed = urlparse(resp.request.url)
    164 +           redirect_parsed = urlparse(url)
    165 +
    166 +           if original_parsed.hostname != redirect_parsed.hostname:
    167 +               try:
    168 +                   del headers['Authorization']
    169 +               except KeyError:
    170 +                   pass
    171 +
    172 +           # However, .netrc might have more auth for us. Let's get it if it
    173 +           # does.
    174 +           new_auth = get_netrc_auth(url)
    175 +           if new_auth is not None:
    176 +               prepared_request.prepare_auth(new_auth)
    177 +
```

Figure 7: the Contents of Patch ***7ba5a53***

```
Better layout for checking.
master · v3.0.0a1

Filter files...
requests
  sessions.py

1 file changed +3 -5 lines changed
requests/sessions.py
@@ -163,11 +163,9 @@ def resolve_redirects(self, resp, req, stream=False, timeout=None,
163 163             original_parsed = urlparse(resp.request.url)
164 164             redirect_parsed = urlparse(url)
165 165
166     -           if original_parsed.hostname != redirect_parsed.hostname:
167     -               try:
168     -                   del headers['Authorization']
169     -               except KeyError:
170     -                   pass
    166 +           if (original_parsed.hostname != redirect_parsed.hostname and
    167 +                   'Authorization' in headers):
    168 +               del headers['Authorization']
```

Figure 8: the Contents of Patch ***326a22e***

In this way, the header is only removed when both the host changes and the header exist, avoiding unnecessary exception handling, simplifying the code structure and improving readability and efficiency. Patch ***7ba5a53*** already sufficiently fixes the vulnerability, and Patch ***326a22e*** just provides a cleaner implementation as a better layout.

Thus, we label the relationship between the two patches of this CVE as *Better Solution*.

========= *Fixing-of-Fixing* Relationship =========

1 The example vulnerability and its patches

1.1 The description for the vulnerability

This example is a vulnerability item (CVE-2019-12108), which documents a denial-of-service (DoS) vulnerability in the *MiniUPnP* software suite. This vulnerability is caused by a null pointer dereference flaw in the affected code. When processing specially crafted network packets or protocol messages, the program incorrectly attempts to access memory through a null pointer, leading to an unexpected program crash. The description is as follows:

NIST

Information Technology Laboratory

NATIONAL VULNERABILITY DATABASE

VULNERABILITIES

CVE-2019-12108 Detail

Description

A Denial Of Service vulnerability in MiniUPnP MiniUPnPd through 2.1 exists due to a NULL pointer dereference in GetOutboundPinholeTimeout in upnpsoap.c for int_port.

1.2 The patches of this vulnerability

As indicated by the NVD Platform[2], Commit ***13585f1*** and Commit ***86030db*** are two patches of this vulnerability:

- ***13585f1****5c7f7dc28bbbba1661efb280d530d114c*
- ***86030db****849260dd8fb2ed975b9890aef1b62b692*

2 The Relationship between Patches

The initial Patch ***13585f1*** introduces a parameter validation strategy in the

function *GetOutboundPinholeTimeout* to address a potential null pointer dereference, but it mistakenly validates *ext_port* instead of the retrieved *rem_port*, leading to false rejection of valid requests, as shown in Figure 9. The subsequent Patch ***86030db*** corrects this bug by replacing *ext_port* with *rem_port* in the conditional check, to fix the false rejection while retaining the original security validation strategy, as shown in Figure 10.

```
GetOutboundPinholeTimeout: check args
master · miniupnpd_2_3_9 ··· minissdpd_1_6_0

Filter files...
miniupnpd
upnpsoap.c

1 file changed +7 -0 lines changed
miniupnpd/upnpsoap.c
@@ -1850,6 +1850,13 @@ GetOutboundPinholeTimeout(struct upnphttp * h, const char * action, const char *
1850  1850        rem_port = GetValueFromNameValueList(&data, "RemotePort");
1851  1851        protocol = GetValueFromNameValueList(&data, "Protocol");
1852  1852
      1853  +     if (!int_port || !ext_port || !protocol)
      1854  +     {
      1855  +         ClearNameValueList(&data);
      1856  +         SoapError(h, 402, "Invalid Args");
      1857  +         return;
      1858  +     }
      1859  +
1853  1860        rport = (unsigned short)atoi(rem_port);
1854  1861        iport = (unsigned short)atoi(int_port);
1855  1862        /*proto = atoi(protocol);*/
```

Figure 9. the Contents of Patch ***13585f1***

```
fix error from commit 13585f1
master · miniupnpd_2_3_9 ··· minissdpd_1_6_0

Filter files...
miniupnpd
upnpsoap.c

1 file changed +1 -1 lines changed
miniupnpd/upnpsoap.c
@@ -1850,7 +1850,7 @@ GetOutboundPinholeTimeout(struct upnphttp * h, const char * action, const char *
1850  1850        rem_port = GetValueFromNameValueList(&data, "RemotePort");
1851  1851        protocol = GetValueFromNameValueList(&data, "Protocol");
1852  1852
1853        -     if (!int_port || !ext_port || !protocol)
      1853  +     if (!int_port || !rem_port || !protocol)
1854  1854        {
1855  1855            ClearNameValueList(&data);
1856  1856            SoapError(h, 402, "Invalid Args");
```

Figure 10. the Contents of Patch ***86030db***

The second patch specifically corrects an error introduced by the first patch rather than independently addressing the original vulnerability.

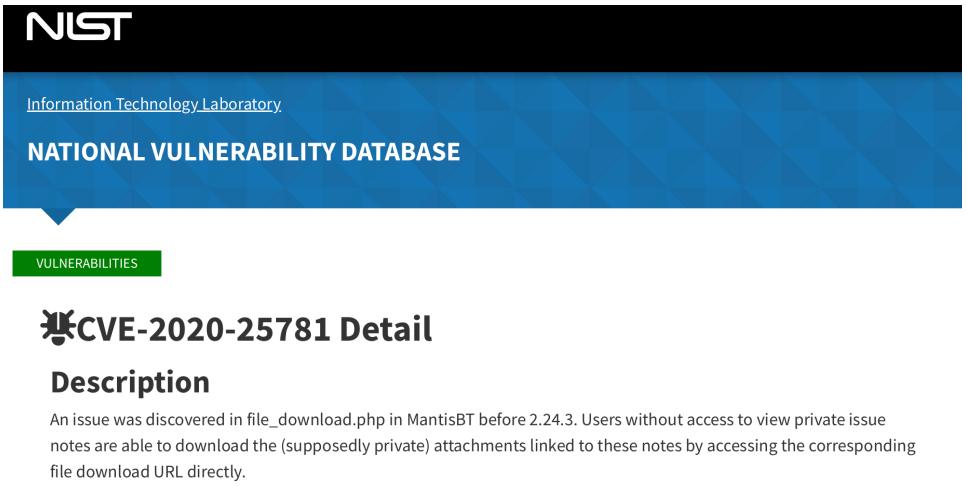
Thus, we label the relationship between the two patches of this CVE as *Fixing-of-Fixing*.

========= *Collaboration* Relationship =========

1 The example vulnerability and its patches

1.1 The description for the vulnerability

This example is a vulnerability item (CVE-2020-25781), which reports a permission bypass flaw in an issue tracking platform: due to insufficient access control checks on file attachments, unauthorized users can download files attached to private issue notes by directly accessing their URLs, even though they are not allowed to view the notes themselves, thus exposing sensitive information contained in private issues. The description is as follows:



1.2 The patches of this vulnerability

As indicated by the NVD Platform[3], Commit *5595c90* and Commit *9de20c0* are two patches of this vulnerability:

- ***5595c90****f11c48164331a20bb9c66098980516e93*
- ***9de20c0****9e5a557e57159a61657ce62f1a4f578fe*

2 The Relationship between Patches

The initial Patch ***5595c90*** establishes the access control mechanism by introducing the *file_can_view_bugnote_attachments*() function, which implements bugnote-level access control checks as shown in Figure 11. Building on this, the subsequent Patch ***9de20c0***

completes the remediation by integrating this new permission check directly into the file download workflow, as shown in Figure 12, ensuring that only users with proper authorization could access the attached files. The second patch relies on the access-control mechanism introduced by the first patch to enforce the intended security policy.

```
+ function file_can_view_bugnote_attachments( $p_bugnote_id, $p_uploader_user_id = null ) {
+     if( $p_bugnote_id == 0 ) {
+         return true;
+     }
+     $t_bug_id = bugnote_get_field( $p_bugnote_id, 'bug_id' );
+     return file_can_view_or_download( 'view', $t_bug_id, $p_uploader_user_id );
+ }
+
```

Figure 11: the Partial Code Changes of Patch ***5595c90***

```
+         if( !file_can_view_bug_attachments( $p_bug_id, $t_user_id )
+         || !file_can_view_bugnote_attachments( $t_attachment_note_id, $t_user_id )
+         ) {
              continue;
```

Figure 12: the Partial Code Changes of Patch ***9de20c0***

Together, the two patches form a complete fix for the vulnerability, where the first patch introduces the checking mechanism and the second patch applies it to the actual file access path.

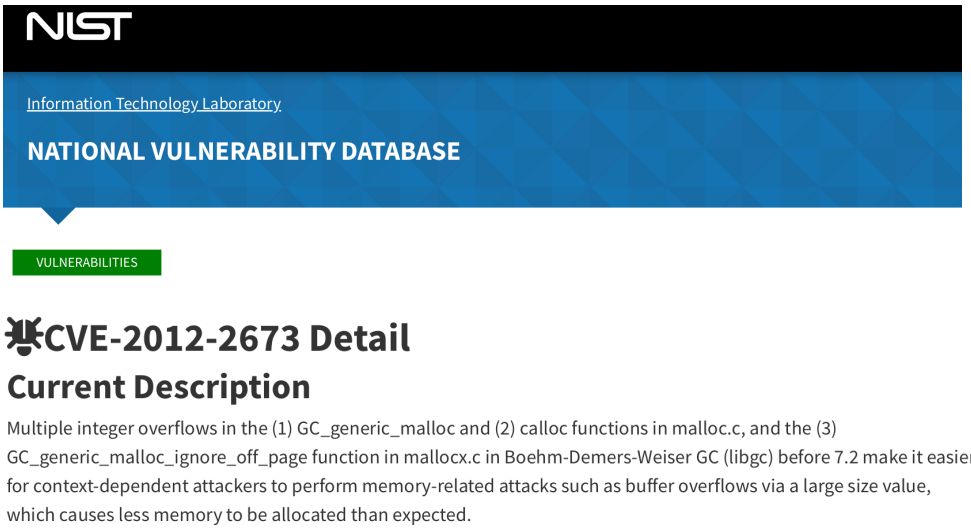
Thus, we label the relationship between the two patches of this CVE as *Collaboration*.

========= *Separation* Relationship =========

1 The example vulnerability and its patches

1.1 The description for the vulnerability

This example is a vulnerability item (CVE-2012-2673), which discloses multiple integer overflow flaws in memory allocation functions. This vulnerabilitie arise when calculating the size of memory to be allocated, where improper handling of large input values can cause integer values to wrap around, resulting in an insufficient memory block being allocated. The description is as follows:



1.2 The patches of this vulnerability

As indicated by the NVD Platform[4], Commit *be9df82* and Commit *e10c1eb* are two patches of this vulnerability:

- ***be9df82***919960214ee4b9d3313523bff44fd99e1
- ***e10c1eb***9908c2774c16b3148b30d2f3823d66a9a

2 The Relationship between Patches

Patch ***be9df82*** addresses allocation size overflows, which is caused by rounding operations in generic memory allocation functions, by adding checks to detect invalid rounded sizes, as shown in Figure 13. Patch ***e10c1eb*** specifically fixes an integer overflow in the *calloc*() implementation by validating that the product of element count and size does not exceed *SIZE_MAX*, as shown in Figure 14.

```
2 files changed +4 -0 lines changed                                   Search within code

malloc.c                                                                                  +2
@@ -169,6 +169,8 @@ GC_API void * GC_CALL GC_generic_malloc(size_t lb, int k)
169 169        GC_bool init;
170 170        lg = ROUNDED_UP_GRANULES(lb);
171 171        lb_rounded = GRANULES_TO_BYTES(lg);
    172 +      if (lb_rounded < lb)
    173 +          return((*GC_get_oom_fn())(lb));
172 174        n_blocks = OBJ_SZ_TO_BLOCKS(lb_rounded);
173 175        init = GC_obj_kinds[k].ok_init;
174 176        LOCK();

mallocx.c                                                                                 +2
@@ -182,6 +182,8 @@ GC_INNER void * GC_generic_malloc_ignore_off_page(size_t lb, int k)
182 182        return(GC_generic_malloc((word)lb, k));
183 183    lg = ROUNDED_UP_GRANULES(lb);
184 184    lb_rounded = GRANULES_TO_BYTES(lg);
    185 +  if (lb_rounded < lb)
    186 +      return((*GC_get_oom_fn())(lb));
185 187    n_blocks = OBJ_SZ_TO_BLOCKS(lb_rounded);
186 188    init = GC_obj_kinds[k].ok_init;
187 189    if (EXPECT(GC_have_errors, FALSE))
```

Figure 13: the Contents of Patch ***be9df82***

```
1 file changed +5 -0 lines changed                                    Search within code

malloc.c                                                                                  +5
@@ -372,8 +372,13 @@ void * malloc(size_t lb)
372 372    }
373 373  #endif /* GC_LINUX_THREADS */
374 374
    375 + #ifndef SIZE_MAX
    376 + #define SIZE_MAX (~(size_t)0)
    377 + #endif
375 378  void * calloc(size_t n, size_t lb)
376 379  {
    380 +    if (lb && n > SIZE_MAX / lb)
    381 +      return NULL;
377 382  #   if defined(GC_LINUX_THREADS) /* && !defined(USE_PROC_FOR_LIBRARIES) */
378 383        /* libpthread allocated some memory that is only pointed to by  */
379 384        /* mmapped thread stacks.  Make sure it's not collectable.      */
```

Figure 14: the Contents of Patch ***e10c1eb***

These two patches address distinct sub-issues of the same vulnerability class by modifying different functions.

Thus, we label the relationship between the two patches of this CVE as *Separation*.

Reference

[1] Xu C, Chen B, Lu C, et al. Tracking patches for open source software vulnerabilities[C]//Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2022: 860-871.

[2] https://nvd.nist.gov/vuln/detail/CVE-2019-12108

[3] https://nvd.nist.gov/vuln/detail/CVE-2020-25781

[4] https://nvd.nist.gov/vuln/detail/CVE-2012-2673